

Daily Progress & Next Steps Plan

Constraint-Interaction Metric for Symbolic Regression

Biswajyoti Nath

Subinoy Nath

July 16, 2026

Part 1: Summary of Today's Work (PySR Integration & Experiments)

Today's focus was implementing and verifying the symbolic regression experimental benchmarking module in `src/experiments.py` and its unit tests in `tests/test_experiments.py`. This establishes the $S(i, j)$ data generation layer.

- **Layer 1 – Dataset Loaders:** Implemented deterministic loaders (`load_feynman_ke`, `load_feynman_coulomb`, `load_polynomial`, `load_srsd_dummy`) with an 80/20 train/test split. Seed-locked dataset splits isolate search randomness from partition variance.
- **Layer 2 – PySR Kwargs Mapper & Composer:** Designed and implemented `build_pysr_kwargs` to map active constraints (C1–C4) dynamically to PySR 1.5.10 regressor parameters. Composed `nested_constraints` (trig nesting depth), `maxdepth`, whitelisted operators (by set intersection), and `elementwise_loss` custom Julia-based penalties additively.
- **Layer 3 – Scenario Runner:** Implemented `run_scenario` to run serial deterministic PySR fits, record search costs (via wall-clock timing as the v1.5.10 proxy), calculate recovery metrics (MSE, relative MSE, NED), and run post-hoc constraint checks on the best expression.
- **Layer 4 – Matrix Orchestrator:** Implemented `run_experiment_matrix` to execute the $4 \times 12 \times 5 = 240$ matrix, save run environment hashes/git metadata in `metadata.json`, and log results incrementally to a run-specific CSV file.
- **Testing:** Added 23 new tests in `tests/test_experiments.py` covering loader shapes/determinism, ground-truth recovery precision, kwargs mapping/merging, and Levenshtein fallback. The entire test suite has **83 passing tests**.
- **Orchestrator Validation:** Successfully ran a mini-matrix integration test, verifying that `metadata.json` captures the environment correctly and the results CSV writes and flushes row-by-row.

Part 2: Strict Plan for Tomorrow (Analysis & Validation)

With raw $M(i, j)$ (density estimates) and $S(i, j)$ (search costs) data generators fully complete, tomorrow's plan is to tie the components together to validate the project's primary acceptance criterion.

- **Phase 1 – Implement `src/analysis.py`:** Build the analysis module to:

- Load the raw `pysr_results.csv` files.
 - Compute $S(i, j) = \min(S_i, S_j)/S_{ij}$ from observed wall-clock times.
 - Exclude timeout-censored runs from $S(i, j)$ calculations using the `timeout_hit` column.
 - Read $M(i, j)$ matrices from the Monte Carlo runs.
 - Calculate Pearson r and p -values between $M(i, j)$ and $S(i, j)$.
- **Phase 2 – Acceptance Verification:** Run the full analysis matrix and check if Pearson $r > 0.6$ and $p < 0.05$ holds across the benchmarks, completing the core acceptance criterion.
 - **Phase 3 – Visualisation:** Generate scatter plots of $M(i, j)$ vs. $S(i, j)$ and heatmap visualizations of the interaction matrix.